

Khant Thu Aung / 105292912

5.3C

ExtensionMethods.cs

```
using SplashScreenSDK;

using System;

using System.IO;

namespace ShapeDrawer
{
    public static class ExtensionMethod
    {
        public static int ReadInteger(this StreamReader reader)
        {
            return Convert.ToInt32(reader.ReadLine());
        }

        public static float ReadSingle(this StreamReader reader)
        {
            return Convert.ToSingle(reader.ReadLine());
        }

        public static Color ReadColor(this StreamReader reader)
        {
            return Color.RGBColor(reader.ReadSingle(), reader.ReadSingle(),
reader.ReadSingle());
        }

        public static void WriteColor(this StreamWriter writer, Color clr)
        {

```

```

        writer.WriteLine("{0}\n{1}\n{2}",clr.R,clr.G,clr.B);
    }
}
}

```

Drawing.cs

```

using System.ComponentModel.Design;
using System.IO.Pipelines;
using SplashKitSDK;

namespace ShapeDrawer
{
    public class Drawing
    {
        private readonly List<Shape> _shapes;
        private Color _background;

        public Drawing(Color background)
        {
            _shapes = new List<Shape>();
            _background = background;
        }

        public Drawing() : this(Color.White)
        {

        }

        public Color Background
    }
}

```

```
{  
    get { return this._background; }  
    set { this._background = value; }  
}  
public int ShapeCount  
{  
    get { return this._shapes.Count; }  
}  
public void AddShape(Shape s)  
{  
    _shapes.Add(s);  
}  
public void RemoveShape(Shape s)  
{  
    _ = _shapes.Remove(s);  
}  
public void Draw()  
{  
    SplashKit.ClearScreen(_background);  
    foreach (Shape s in _shapes)  
    {  
        s.Draw();  
    }  
}  
public void Save(string filename)
```

```

{
    StreamWriter writer;
    writer = new StreamWriter(filename);
    writer.WriteColor(Background);
    writer.WriteLine(_shapes.Count);
    foreach (Shape s in _shapes)
    {
        s.SaveTo(writer);
    }
    writer.Close();
}

public void Load(string filename)
{
    StreamReader reader = new StreamReader(filename);
    try
    {
        Background = reader.ReadColor();
        int count = reader.ReadInteger();
        Shape s;
        _shapes.Clear();
        for (int i = 0; i < count; i++)
        {
            string kind = reader.ReadLine();

            switch (kind)
            {

```

```

        case "Rectangle":
            s = new MyRectangle();
            break;
        case "Circle":
            s = new MyCircle();
            break;
        case "Line":
            s = new MyLine();
            break;
        default:
            throw new InvalidDataException("Unknown Shape kind: " + kind);

    }

    s.LoadFrom(reader);
    AddShape(s);
}

}

finally
{
    reader.Close();
}

}

public void SelectShapesAt(Point2D pt)
{
    foreach (Shape s in _shapes)
    {

```

```

        s.Selected = s.isAt(pt);
    }
}

public List<Shape> SelectedShapes
{
    //read-only
    get
    {
        List<Shape> selectedShapes = new List<Shape>();
        foreach (Shape s in _shapes)
        {
            if (s.Selected) selectedShapes.Add(s);
        }
        return selectedShapes;
    }
}
}

```

MyRectangle.cs

```

using SplashKitSDK;

namespace ShapeDrawer
{
    public class MyRectangle : Shape
    {
        private int _width;
        private int _height;
    }
}

```

```
/// Constructors
```

```
public MyRectangle(Color color, int x, int y, int width, int height) : base(color)
```

```
{
```

```
    X = x;
```

```
    Y = y;
```

```
    Width = width;
```

```
    Height = height;
```

```
}
```

```
public MyRectangle() : this(Color.Green, 0, 0, 112, 112) {}
```

```
/// Properties
```

```
public int Width
```

```
{
```

```
    get { return _width; }
```

```
    set { _width = value; }
```

```
}
```

```
public int Height
```

```
{
```

```
    get { return _height; }
```

```
    set { _height = value; }
```

```
}
```

```
public override void Draw()
```

```
{
```

```
    if (Selected)
```

```
    {
```

```
        DrawOutline();
```

```
    }
```

```

        SplashKit.FillRectangle(Color, X, Y, _width, _height);
    }

    public override void DrawOutline()
    {
        int outlineWidth = 7;

        SplashKit.FillRectangle(Color.Black, X - outlineWidth, Y - outlineWidth, _width +
outlineWidth * 2, _height + outlineWidth * 2);
    }


    public override bool isAt(Point2D pt)
    {
        return (pt.X >= X && pt.X <= X + _width && pt.Y >= Y && pt.Y <= Y + _height);
    }

    public override void SaveTo(StreamWriter writer)
    {
        writer.WriteLine("Rectangle");

        base.SaveTo(writer);

        writer.WriteLine(Width);

        writer.WriteLine(Height);
    }

    public override void LoadFrom(StreamReader reader)
    {
        base.LoadFrom(reader);

        Width = reader.ReadInteger();

        Height = reader.ReadInteger();
    }

```



```
}  
}
```

MyCircle.cs

```
using SplashKitSDK;  
  
namespace ShapeDrawer  
{  
    public class MyCircle : Shape  
    {  
        private int _radius;  
  
        public MyCircle() : this(Color.Blue, 62)  
        {  
        }  
  
        public MyCircle(Color color, int radius) : base(color)  
        {  
            _radius = radius;  
        }  
  
        /// Properties  
  
        public int Radius  
        {  
            get { return _radius; }  
            set { _radius = value; }  
        }  
  
        //overridden methods  
  
        public override void Draw()  
        {  
            if (Selected)
```

```

    {
        this.DrawOutline();
    }

    SplashKit.FillCircle(Color, X, Y, _radius);
}

public override void DrawOutline()
{
    int outlineWidth = 2;

    SplashKit.FillCircle(Color.Black, X, Y, Radius + outlineWidth);
}

public override bool isAt(Point2D pt)
{
    // return (pt.X >= X - Radius && pt.X <= X + Radius && pt.Y >= Y - Radius && pt.Y <= Y +
Radius);

    Circle c = SplashKit.CircleAt(X, Y, Radius);

    return SplashKit.PointInCircle(pt, c);
}

public override void SaveTo(StreamWriter writer)
{
    writer.WriteLine("Circle");

    base.SaveTo(writer);

    writer.WriteLine(Radius);
}

public override void LoadFrom(StreamReader reader)
{
    base.LoadFrom(reader);
}

```

```

        Radius = reader.ReadInteger();
    }
}
}

```

MyLine.cs

```

using System.Runtime.Intrinsics.X86;
using SplashKitSDK;
namespace ShapeDrawer
{
    public class MyLine : Shape
    {
        private float _endX;
        private float _endY;
        public MyLine() : this(Color.Red)
        {
        }
        public MyLine(Color color):base(color)
        {
            X = SplashKit.MouseX();
            Y = SplashKit.MouseY();
            _endX = X + 200;
            _endY = Y;
        }
        //properties
        public float EndX
        {

```

```

    get {return _endX;}

    set {_endX = value;}
}

public float EndY
{
    get {return _endY;}

    set {_endY = value;}
}

//overridden methods
public override void Draw()
{
    if (Selected)
    {
        DrawOutline();
    }

    SplashKit.DrawLine(Color, X, Y, _endX, _endY);
}

public override void DrawOutline()
{
    int radius = 3;

    SplashKit.DrawCircle(Color.Black, X, Y, radius); //start circle

    SplashKit.DrawCircle(Color.Black, _endX, _endY, radius); //end circle
}

public override bool isAt(Point2D pt)
{
    Line line = SplashKit.LineFrom(X, Y, _endX, _endY);

```

```

        return SplashKit.PointOnLine(pt, line, 5);
    }

    public override void SaveTo(StreamWriter writer)
    {
        writer.WriteLine("Line");
        base.SaveTo(writer);
        writer.WriteLine(EndX);
        writer.WriteLine(EndY);
    }

    public override void LoadFrom(StreamReader reader)
    {
        base.LoadFrom(reader);
        EndX = reader.ReadInteger();
        EndY = reader.ReadInteger();
    }
}

```

Program.cs

```

using System;
using SplashKitSDK;

namespace ShapeDrawer
{
    public class Program
    {
        private enum ShapeKind

```

```

{
    Rectangle,
    Circle,
}

public static void Main()
{
    Window window = new Window("Shape Drawer", 800, 600);

    Drawing myDrawing = new Drawing();

    ShapeKind kindToAdd = ShapeKind.Circle;

    do
    {
        SplashKit.ProcessEvents();

        SplashKit.ClearScreen();

        if(SplashKit.KeyTyped(KeyCode.RKey))
        {
            kindToAdd = ShapeKind.Rectangle;
        }

        if(SplashKit.KeyTyped(KeyCode.CKey))
        {
            kindToAdd = ShapeKind.Circle;
        }

        if (SplashKit.MouseClicked(MouseButton.LeftButton))
        {
            Shape newShape;

```

```

switch (kindToAdd)
{
    case ShapeKind.Circle:
        newShape = new MyCircle();
        break;
    default:
        newShape = new MyRectangle();
        break;
}

newShape.X = SplashKit.MouseX();
newShape.Y = SplashKit.MouseY();
myDrawing.AddShape(newShape);
}

if (SplashKit.KeyTyped(KeyCode.SpaceKey))
{
    myDrawing.Background = SplashKit.RandomColor();
}

if (SplashKit.MouseClicked(MouseButton.RightButton))
{
    myDrawing.SelectShapesAt(SplashKit.MousePosition());
}

if (SplashKit.KeyTyped(KeyCode.DeleteKey) ||
    SplashKit.KeyTyped(KeyCode.BackspaceKey))
{
    foreach (Shape s in myDrawing.SelectedShapes)

```

```

    {
        myDrawing.RemoveShape(s);
    }
}

if (SplashKit.KeyTyped(KeyCode.LKey))
{
    int count = 2; //studentID= 105292912

    for (int i = 0; i < count; i++)
    {
        float startX = SplashKit.MouseX();
        float startY = SplashKit.MouseY() + i * 40;

        MyLine newLine = new MyLine();
        newLine.X = startX;
        newLine.Y = startY;
        newLine.EndX = startX + 200;
        newLine.EndY = startY; // horizontal line

        myDrawing.AddShape(newLine);
    }

}

if(SplashKit.KeyTyped(KeyCode.SKey))
{

```



```

        //saving total 7 shapes

        myDrawing.Save("TestDrawing.txt"); //id = 105292912 -> 5+X -> X = 5%2 = 2 -> 5+2
= 7

    }
    if(SplashKit.KeyTyped(KeyCode.OKey))
    {
        try
        {
            myDrawing.Load("TestDrawing.txt");
        }catch(Exception e)
        {
            Console.Error.WriteLine("Error loading file:{0}",e.Message);
        }
        // myDrawing.Load("TestDrawings.txt");
    }
    myDrawing.Draw();
    SplashKit.RefreshScreen();
} while (!window.CloseRequested);
}
}
}

```



